

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_04_)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B__

Date performed: _9th September _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task we are required to do arithmetic operations on RISC-V while using the Venus simulator so the variables and their values are provided so we initialize it in .data where it will be stored and executable and write code in globl main where I performed the given operations firstly load the word by using lw in the register then performed the operations by using add sub mul and storing the output in the storing register then assemble it on the simulator and then see step by step how values are storing in the registers after the final run the stored values in the registers and in decimal form. Also at the end of the code we ecall it.

Code:

Code1:

.data

A: .word 5

B: .word 4

.text

.globl main

lw t0, A

lw t1, B

add s0, t0, t1

sub s1, t0, t1

mul s2, t0, t1

mul s3, t0, t0

li a7, 93

li a0, 0

ecall

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	lw t0 , A
0x4	0x0002A283	lw x5 0(x5)	lw t0 , A
0x8	0x10000317	auipc x6 65536	lw t1 , B
0xc	0xFFC32303	lw x6 -4(x6)	lw t1 , B
0x10	0x00628433	add x8 x5 x6	add s0, t0, t1#s0=9 C
0x14	0x406284B3	sub x9 x5 x6	sub s1, t0, t1#s1=1 D
0x18	0x02628933	mul x18 x5 x6	mul s2, t0, t1#s2=20 E
0x1c	0x025289B3	mul x19 x5 x5	mul s3, t0, t0#s3=25 F
0x20	0x40828A33	sub x20 x5 x8	sub s4, t0, s0 #a-c
0x24	0x01248AB3	add x21 x9 x18	add s5, s1, s2#d+e
0x28	0x033A8B33	mul x22 x21 x19	mul s6, s5, s3#(d+e)*f
0x2c	0x016A0BB3	add x23 x20 x22	add s7, s4, s6
0x30	0x05D00893	addi x17 x0 93	li a7, 93
0x34	0x00000513	addi x10 x0 0	li a0, 0
0x38	0x00000073	ecall	ecall

t0 (x5)	5	s0 (x8)	9
t1 (x6)	4	s1 (x9)	1
a7 (x17)	93		
s2 (x18)	20		
s3 (x19)	25		

CONVERSION OF MACHINE CODE TO DECIMAL FORM:

MACHINE CODE	DECIMAL FORM
0x10000297	268,436,119
0x0002A283	172,675
0x00628433	6,456,371
0x406284B3	1,080,200,371
0x02628933	39,992,371

0x025289B3	38,996,403
0x40828A33	1,082,298,931
0x01248AB3	19,172,019
0x033A8B33	54,168,371
0x016A0BB3	23,727,027
0x05D00893	97,519,763
0x00000513	1,299
0x00000073	115

Task 2:

Description:

In this task an expression is given that is to be covered in the RISC-V format so this is the task1 extension so we are using the outputs in which we stored the values of E,F,G etc for further applying the arithmetic operations so by applying the all operations are required to make that expression to find m value we did and store in more registers that are for storing then we run the simulation step by step after that we see the final run the values are correctly stored and also in the assemble side the machine code PC original code and the instructions are written that we give for execution.

Code:

.data

A: .word 5

B: .word 4

.text

.globl main

lw t0 , A

lw t1 , B

add s0, t0, t1#s0=9 C

sub s1, t0, t1#s1=1 D

mul s2, t0, t1#s2=20 E

```
mul s3, t0, t0#s3=25 F
```

```
sub s4, t0, s0 #a-c
```

```
add s5, s1, s2#d+e
```

```
mul s6, s5, s3#(d+e)*f
```

```
add s7, s4, s6
```

```
li a7, 93
```

```
li a0, 0
```

```
ecall
```

```
1 .data
2     A: .word 5
3     B: .word 4
4
5 .text
6
7 .globl main
8     lw t0 , A
9     lw t1 , B
0     add s0, t0, t1#s0=9 C
1     sub s1, t0, t1#s1=1 D
2     mul s2, t0, t1#s2=20 E
3     mul s3, t0, t0#s3=25 F
4     sub s4, t0, s0 #a-c
5     add s5, s1, s2#d+e
6     mul s6, s5, s3#(d+e)*f
7     add s7, s4, s6
8     li a7, 93
9     li a0, 0
0     ecall
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	lw t0 , A
0x4	0x0002A283	lw x5 0(x5)	lw t0 , A
0x8	0x10000317	auipc x6 65536	lw t1 , B
0xc	0xFFC32303	lw x6 -4(x6)	lw t1 , B
0x10	0x0628433	add x8 x5 x6	add s0, t0, t1#s0=9 C
0x14	0x406284B3	sub x9 x5 x6	sub s1, t0, t1#s1=1 D
0x18	0x02628933	mul x18 x5 x6	mul s2, t0, t1#s2=20 E
0x1c	0x025289B3	mul x19 x5 x5	mul s3, t0, t0#s3=25 F

0x20	0x40828A33	sub x20 x5 x8	sub s4, t0, s0 #a-c
0x24	0x01248AB3	add x21 x9 x18	add s5, s1, s2#d+e
0x28	0x033A8B33	mul x22 x21 x19	mul s6, s5, s3#(d+e)*f
0x2c	0x016A0BB3	add x23 x20 x22	add s7, s4, s6
0x30	0x05D00893	addi x17 x0 93	li a7, 93
0x34	0x00000513	addi x10 x0 0	li a0, 0
0x38	0x00000073	ecall	ecall

tp
(x4)

0

t0
(x5)

5

t1
(x6)

4

s0
(x8)

9

s1
(x9)

1

a7
(x17)

93

s2
(x18)

20

s3
(x19)

25

s4
(x20)

-4

s5
(x21)

21

s6
(x22)

525

s7
(x23)

521

Task 3:

Description:

In this task the two numbers are given that are in hexadecimal form so we initialize them with the name n1 and n2 in .data and in the globl main we write the instructions that are asked to be execute so firstly we load the numbers into the variables t0 and t1 then we write instruction for and or and xor and storing there output into the storing registers, then we run the simulation and see that where the ans were stored in the registers.

Code:

```
.data

n1: .word 0x0fff

n2: .word 0xf0f


.text


.globl main
main:

    lw t0 , n1

    lw t1 , n2


    and s0, t0, t1

    or  s1, t0, t1

    xor s2, t0, t1

    li a7, 93

    li a0, 0

    ecall
```

```

1 .data
2     n1: .word 0x0fff
3     n2: .word 0xf0f
4
5 .text
6
7 .globl main
8 main:
9     lw t0 , n1
10    lw t1 , n2
11
12    and s0, t0, t1
13    or  s1, t0, t1
14    xor s2, t0, t1
15
16    li a7, 93
17    li a0, 0
18    ecall

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	lw t0 , n1
0x4	0x0002A283	lw x5 0(x5)	lw t0 , n1
0x8	0x10000317	auipc x6 65536	lw t1 , n2
0xc	0xFFC32303	lw x6 -4(x6)	lw t1 , n2
0x10	0x0062F433	and x8 x5 x6	and s0, t0, t1
0x14	0x0062E4B3	or x9 x5 x6	or s1, t0, t1
0x18	0x0062C933	xor x18 x5 x6	xor s2, t0, t1
0x1c	0x05D00893	addi x17 x0 93	li a7, 93
0x20	0x00000513	addi x10 x0 0	li a0, 0
0x24	0x00000073	ecall	ecall

t0 (x5)	0x00000FFF
t1 (x6)	0x00000F0F
t2 (x7)	0x00000000
s0 (x8)	0x00000F0F
s1 (x9)	0x00000FFF
a0 (x10)	0x00000000
a7 (x17)	0x0000005D
s2 (x18)	0x000000F0

Task 4:

Description:

The code is provided to us and there are some syntax and logical error that are resolved. Firstly there is an error in .section instead of this we write .data and after loading immediate for starting the loop syntax loop: is used, there is a bge means if l is greater than size so it will exit not break and for the jump back to the loop so loop is used at the end of the instructions and after the loop exit: is used instead of 1 for exiting loop.

Code:

.text

```
.globl arraysum

arraysum:

    # a0 = int a[]

    # a1 = int size

    # t0 = ret

    # t1 = i

    li    t0, 0        # ret = 0

    li    t1, 0        # i = 0

loop:      # For loop (Label '1:' changed to 'loop:')

    bge   t1, a1, exit  # if i >= size, break (Changed '1f' to 'exit')

    slli  t2, t1, 2      # Multiply i by 4 (1 << 2 = 4)

    add   t2, a0, t2     # Update memory address

    lw    t2, 0(t2)      # Dereference address to get integer

    add   t0, t0, t2     # Add integer value to ret

    addi  t1, t1, 1      # Increment the iterator

    j     loop          # Jump back to start of loop (Changed '1b' to 'loop')

exit:      # Loop End Label (Changed '1:' to 'exit:')

    mv    a0, t0         # Move t0 (ret) into a0

    ret                    # Return via return address register
```

```

1 .text
2 .globl arraysum
3 arraysum:
4     # a0 = int a[]
5     # a1 = int size
6     # t0 = ret
7     # t1 = i
8     li    t0, 0
9     li    t1, 0
10 loop:
11     bge   t1, a1, exit
12     slli  t2, t1, 2
13     add   t2, a0, t2
14     lw    t2, 0(t2)
15     add   t0, t0, t2
16     addi  t1, t1, 1
17     j     loop
18 exit:
19     mv    a0, t0
20     ret

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000293	addi x5 x0 0	li t0, 0 # ret = 0
0x4	0x00000313	addi x6 x0 0	li t1, 0 # i = 0
0x8	0x00B35E63	bge x6 x11 28	bge t1, a1, exit # if i >= size, break (Changed 'lf' to 'exit')
0xc	0x00231393	slli x7 x6 2	slli t2, t1, 2 # Multiply i by 4 (1 << 2 = 4)

Task 5:

Description:

In this task we are required to two table code so firstly we initialize it in .data section then give instructions in globl main where we load address of the table in t0 then by using lw we load the values in registers .basically we are doing offset+t0 so that the to generate the table of two. Then we run the simulation and see the readings are stored in the registers.

Code:

.data

table: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20

.text

.globl main

main:

la t0, table

lw s0, 0(t0) # s0 = 2

lw s1, 4(t0) # s1 = 4

lw s2, 8(t0) # s2 = 6

lw s3, 12(t0) # s3 = 8

lw s4, 16(t0) # s4 = 10

lw s5, 20(t0) # s5 = 12

lw s6, 24(t0) # s6 = 14

lw s7, 28(t0) # s7 = 16

lw s8, 32(t0) # s8 = 18

lw s9, 36(t0) # s9 = 20

li a7, 93

li a0, 0

ecall

```

1 .data
2     table: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20
3
4 .text
5 .globl main
6 main:
7     la t0, table
8
9     lw s0, 0(t0)      # s0 = 2
10    lw s1, 4(t0)      # s1 = 4
11    lw s2, 8(t0)      # s2 = 6
12    lw s3, 12(t0)     # s3 = 8
13    lw s4, 16(t0)     # s4 = 10
14    lw s5, 20(t0)     # s5 = 12
15    lw s6, 24(t0)     # s6 = 14
16    lw s7, 28(t0)     # s7 = 16
17    lw s8, 32(t0)     # s8 = 18
18    lw s9, 36(t0)     # s9 = 20
19
20    li a7, 93
21    li a0, 0
22    ecall

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, table
0x4	0x00028293	addi x5 x5 0	la t0, table
0x8	0x0002A403	lw x8 0(x5)	lw s0, 0(t0) # s0 = 2
0xc	0x0042A483	lw x9 4(x5)	lw s1, 4(t0) # s1 = 4
0x10	0x0082A903	lw x18 8(x5)	lw s2, 8(t0) # s2 = 6
0x14	0x00C2A983	lw x19 12(x5)	lw s3, 12(t0) # s3 = 8
0x18	0x0102AA03	lw x20 16(x5)	lw s4, 16(t0) # s4 = 10
0x1c	0x0142AA83	lw x21 20(x5)	lw s5, 20(t0) # s5 = 12

0x20	0x0182AB03	lw x22 24(x5)	lw s6, 24(t0) # s6 = 14
0x24	0x01C2AB83	lw x23 28(x5)	lw s7, 28(t0) # s7 = 16
0x28	0x0202AC03	lw x24 32(x5)	lw s8, 32(t0) # s8 = 18
0x2c	0x0242AC83	lw x25 36(x5)	lw s9, 36(t0) # s9 = 20
0x30	0x05D00893	addi x17 x0 93	li a7, 93
0x34	0x00000513	addi x10 x0 0	li a0, 0
0x38	0x00000073	ecall	ecall

s0 (x8)	2
s1 (x9)	4

s4 (x20)	10	
s5 (x21)	12	
s6 (x22)	14	
s7 (x23)	16	
s8 (x24)	18	
s9 (x25)	20	